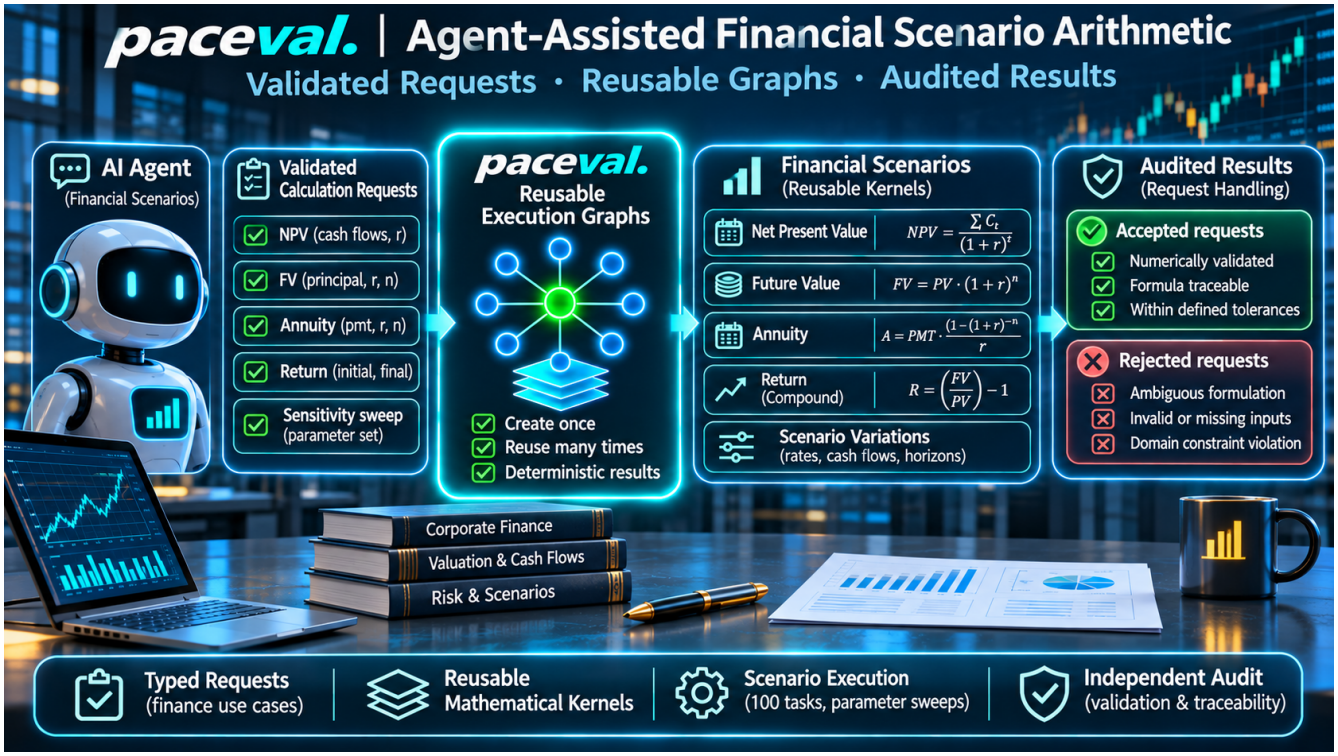
Agent-Assisted Financial Scenario Arithmetic

An executable, independently audited paceval mathematical-tool demonstrator



Cover illustration: agent-facing financial requests pass through typed validation before reusable paceval graph evaluation.

Research outcome. Four predefined financial models were implemented with direct paceval C/C++ SDK calls and a persistent local tool adapter. One hundred valid research tasks and twenty intentionally malformed tasks were executed, with 1,600 sensitivity results checked independently at 100 decimal digits.

Scope. The environment uses synthetic EUR/USD-denominated scenarios and a compatible x86-64 Linux runtime. This document reports computational feasibility and reproducible numeric agreement, not investment advice, transaction execution, a production MCP service, or a validated market model.

1. Research question and execution architecture

The originating WP-011 proposal focused on repeated cash-flow and return arithmetic. This implementation tests whether a typed agent request can be admitted, evaluated with a previously compiled paceval expression graph, repeated over an input sweep, and independently audited without delegating numeric execution to a language model.

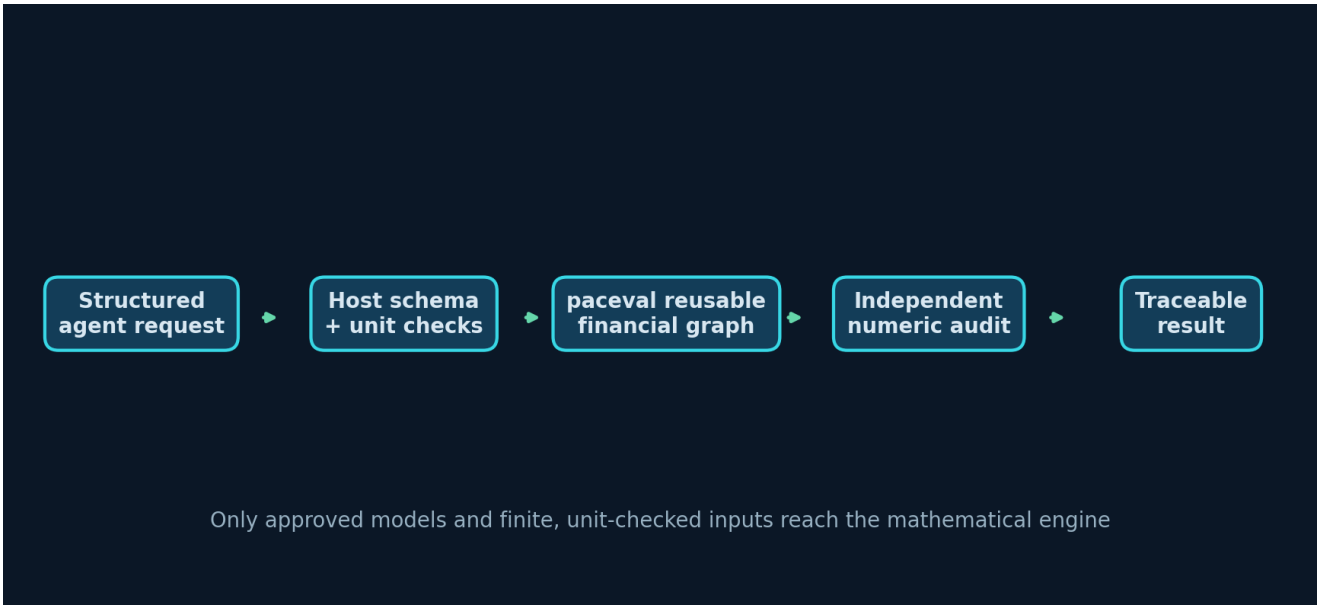


Figure 1. Explicit trust boundary. Host-side policy and domain checks precede the paceval execution graph.

The host rejects missing, non-finite, out-of-range, ambiguous and unapproved inputs; it owns scenario generation, output serialization and independent auditing. paceval owns the mathematical graph creation and evaluation. A graph remains available for subsequent requests rather than rebuilding its formula at every call.

Experimental context: not an end-to-end agent trial. The JSONL adapter demonstrates the callable tool interface with synthetic agent-like requests. There is no connected LLM or remote MCP protocol endpoint.

2. Approved mathematical model contracts

Kernel	Expression and timing convention
Six-period NPV	NPV = Σ(C_t / (1+r)^t), t = 0...5; C_0 is undiscounted; cash flows annual.
Future value	FV = principal × exp(years × ln(1+r)); annual compounding.
Annuity present value	PV = payment × (1 − exp(−years × ln(1+r))) / r; year-end payments.
CAGR	CAGR = exp(ln(final/principal)/years) − 1; decimal annual return.

Each formula is registered under an exact model ID and version. The agent-facing input schema accepts only the parameters needed by its selected model. The C++ SDK interface uses eleven consistently named variables; unused internal slots are populated with fixed values and are not additional user requirements.

Conventions and domain controls.

The unit contract requires one input currency (EUR or USD), annual periods, and rates expressed as decimal per year (for example 0.05 means 5%). Research-domain bounds include rates from 0.001 to 0.30; full years 1–30; strictly positive principal and final value; and finite bounded cash flows. No FX conversion, market-rate discovery, business-case interpretation or automatic parameter estimation takes place.

The original WP-011 expression was notation-level NPV. Here the six-period summation is expanded into an actual expression string accepted by the tested paceval runtime.

3. Deterministic research workload

The dataset comprises 100 valid tasks distributed equally over four mathematical models plus 20 deliberately invalid tasks. Each valid task has 16 deterministic sensitivity scenarios. Rates are varied within an approved annual interval; CAGR tasks vary final values. These are synthetic controlled inputs with predefined assumptions, not real financial contracts.

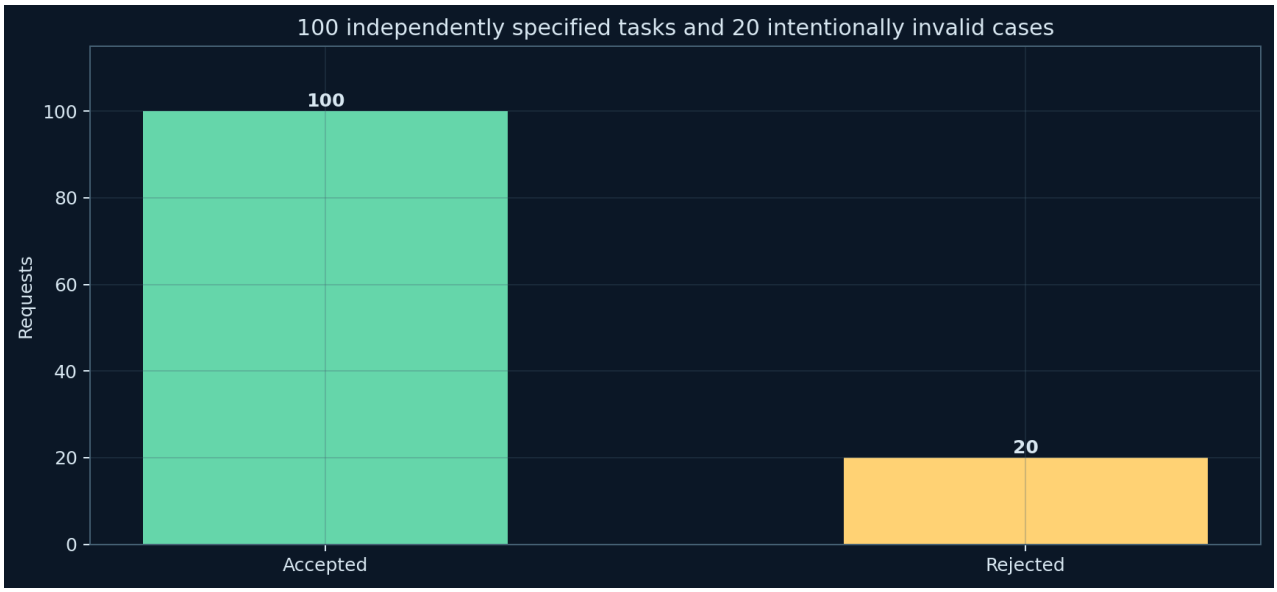


Figure 2. Host validation accepts valid tasks and rejects the deliberately erroneous input classes.

Rejection classes include unapproved model names, unsupported formula versions, missing or ambiguous units, out-of-range rates or time horizons, currency mismatches, and invalid financial-domain values. Each accepted request has a stable ID, model ID, formula version and output unit.

4. Numerical execution and independent audit

Test	Observed result
Batch arithmetic	1,600 / 1,600 scalar values within the pre-defined tolerance
Independent decimal calculation	1,600 / 1,600 checked with 100-digit precision
Controlled version update	16 / 16 changed NPV values independently verified
Native C++ reference	All 1,600 sensitivity samples passed
Tolerance	Absolute difference ≤ $10^{-9} \times (1 + \lvert\text{reference}\rvert)$



Figure 3. Distribution of arithmetic discrepancies relative to a separately coded native C++ calculation.

The maximum absolute distance to the separate 100-digit reference in the released audit is approximately 1.07e-10 in model output units. Accuracy is assessed against the specified mathematical expressions only; it does not imply that any discount rate or cash-flow assumption is economically correct.

5. Sensitivity studies and reusable graphs

A single computation graph is created for each approved mathematical model and reused for multiple tasks and input sweeps. The agent-facing stream keeps all four graphs alive inside one C++ subprocess. In twelve valid JSONL calls, four calls were initial uses and eight calls explicitly returned graph_reused=true.

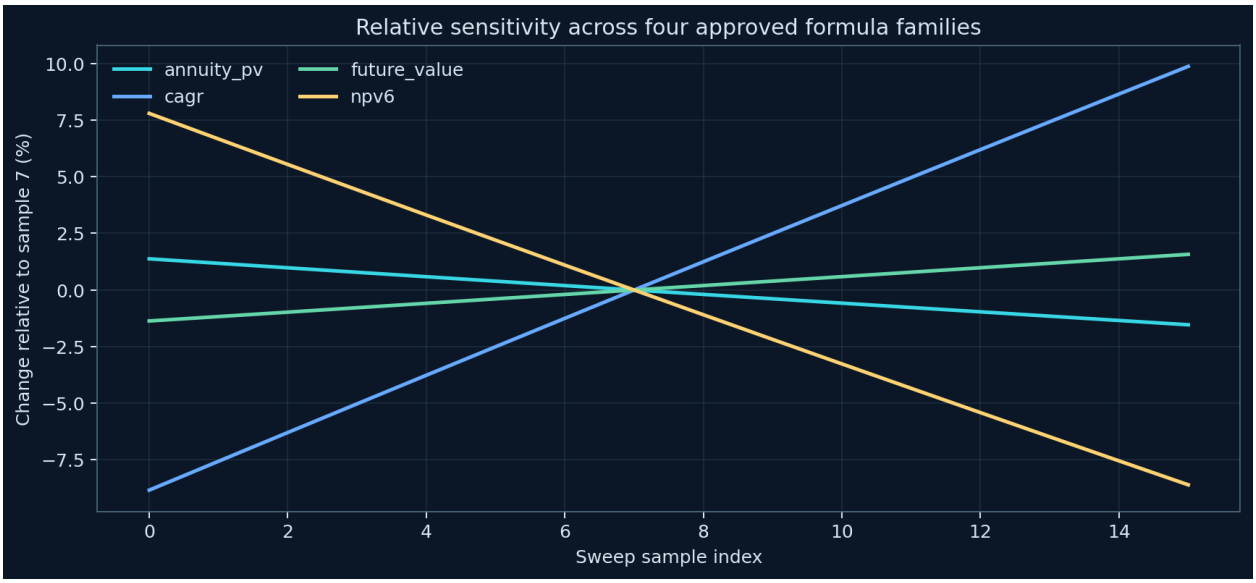


Figure 4. Normalized model sensitivities from the generated, deterministic CSV examples.

Model parameters and interest-rate conventions are fixed within each calculation. Derivative estimation is not part of this study; the plotted values are direct repeated evaluations of the approved formulas.

6. Version control and agent interface

A controlled update changes the final cash-flow term in NPV from C5 to 0.98 × C5, creating NPV v2 as a separate computation graph. All 16 revised outputs are checked against an independent calculation before any future promotion to the agent-facing approved model registry. The original NPV v1 graph remains available and the other three model graphs are not rebuilt.

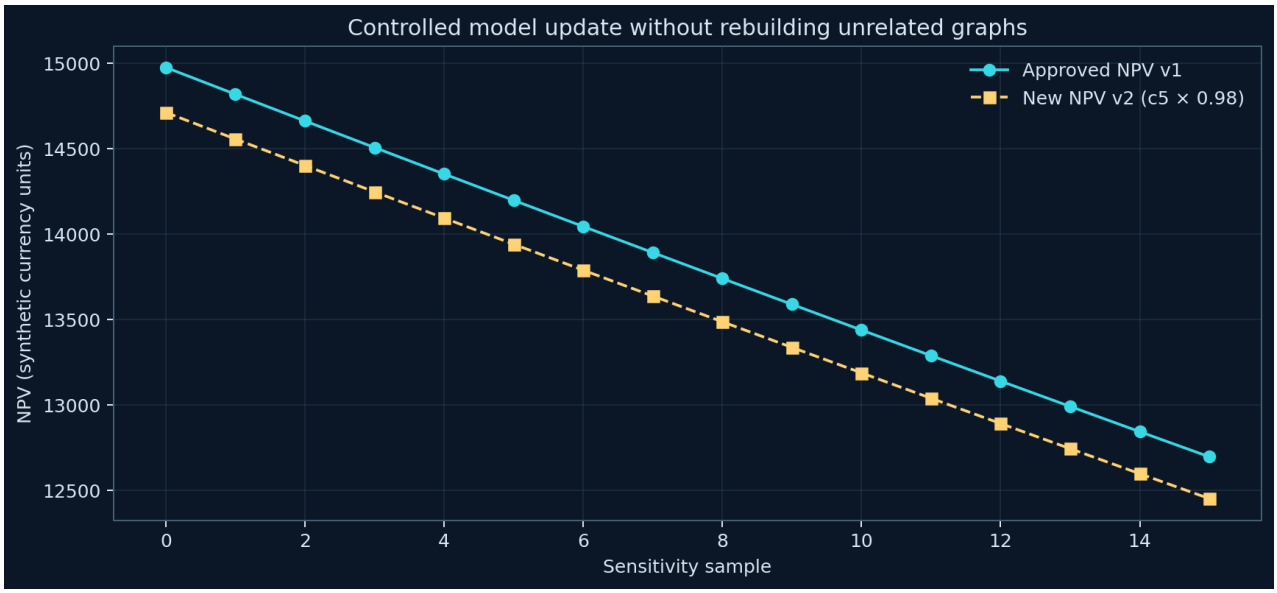


Figure 5. A deliberately different financial assumption creates a traceable new formula version.

The local JSONL tool adapter supports the approved model identifiers and model-specific parameter schemas, with explicit currency, annual rate, year and version fields. It rejects missing parameters and invalid request types before invoking the C++ core. Test responses record numeric value, output unit, status and graph-reuse indicator. This is a tool-contract demonstration, not a remote MCP or LLM deployment.

7. TINC interval evaluation: equivalent mathematical graphs

The baseline one-period discount equation is principal / (1 + rate). Under the tested Linux runtime, its raw TINC intervals included the exact-real reference for only 22/24 fixed samples when the numerical reference was evaluated from the actual IEEE-754 binary64 inputs. The older 23/24 count was based on the displayed decimal CSV strings, and did not fully represent the binary inputs.

For the validated input domain rate > -1, the mathematically identical expression principal * exp(-ln(1 + rate)) provides an alternative computation graph. The original paceval library returned raw, unmodified TINC intervals that enclosed the independent 100-digit reference in 24/24 fixed samples and 1,500/1,500 additional deterministic samples. No host-side interval enlargement was used.

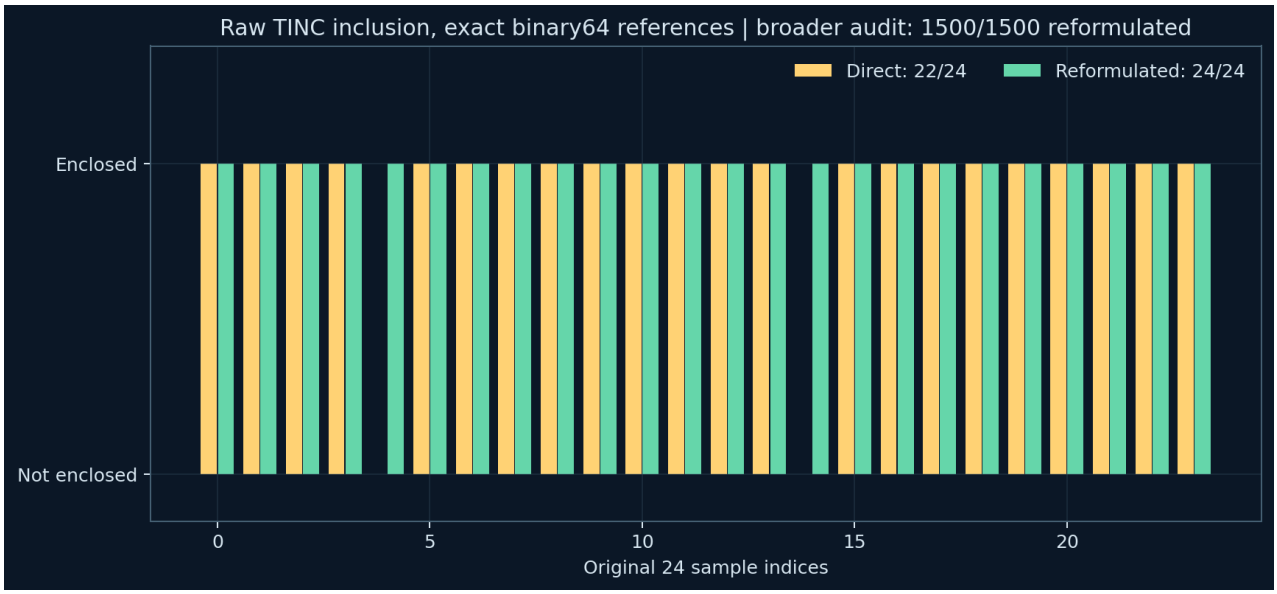


Figure 6. Same mathematical discount function: direct-division TINC vs. reformulated TINC, against the exact input binary64 values.

The separate sample audit reproduces both raw result sets, preserves the baseline non-inclusions, and fails automatically if any reformulated interval misses its reference. The numerical inclusion observation is specific to the tested version, input domain and samples; it is not a universal interval-proof claim. The primary NPV, annuity, future-value and CAGR evaluations are independently checked as before.

8. Reproducibility and scope

Build: Compile src/financial_agent.cpp and the unmodified official public API facade src/paceval_main.cpp with CMake, linking a compatible external Linux .so. The published repository intentionally contains no shared library and no LICENSE_NOTICE.md. The original SDK copyright and license notices remain in its two source files.

Run: ctest from the build directory; run financial_agent --input examples/requests.csv --batch 37 --out reproduced; run python3 src/high_precision_reference.py reproduced; invoke src/agent_tool_adapter.py against the persistent C++ process with the included JSONL cases. All generated scalar results, model versions and rejection reasons can be traced to the included CSV/JSONL examples.

Responsibility	Implemented by
Graph creation and numeric evaluation	Official paceval C/C++ API
Validation, sensitive-data policy, scenario sweeps	C++ host and local Python JSONL adapter
High-precision reference and charts	Separate Python Decimal and Matplotlib scripts
Cash-flow assumptions, rates and investment decisions	Externally supplied; not generated or endorsed by this tool

9. External references

1. paceval mathematical engine, public C/C++ SDK and computation API: https://paceval.com/api/

2. Model Context Protocol specification and typed tool contracts: https://modelcontextprotocol.io/specification/2025-11-25/server/tools

3. OWASP LLM01 Prompt Injection and trust-boundary recommendations: https://genai.owasp.org/llmrisk/llm01-prompt-injection/

4. U.S. SEC discussion of discounted-cash-flow valuation methodology: https://www.sec.gov/Archives/edgar/vprr/0800/08002077.pdf

Conclusion: The tested architecture successfully integrates reusable paceval mathematical graphs into a typed synthetic financial-scenario workflow with numerical verification and traceable rejections. Production-grade financial suitability, market-data validity, financial advice, and full agent integration are beyond the demonstrated scope.